

Tài liệu kỹ thuật Sonic Math Runner

1. Mục tiêu tài liệu

Tài liệu này được viết lại dựa trên 2 nguồn:

- Mã nguồn hiện có trong repo `Game-Sonic-Running`
- Báo cáo `BaoCao_SonicMathRunner.docx`

Mục tiêu là mô tả rõ cách tạo ra trò chơi này theo góc nhìn kỹ thuật: chọn công nghệ gì, cấu trúc hệ thống ra sao, dùng công cụ nào để lập trình, làm dữ liệu, dựng nhân vật, quản trị câu hỏi, đóng gói PWA và triển khai.

2. Kết luận nhanh

`Sonic Math Runner` hiện tại là một game web 3D kiểu endless runner kết hợp quiz Toán học, chạy trực tiếp trên trình duyệt, có backend Node.js để cấp dữ liệu câu hỏi và trang admin để chỉnh ngân hàng câu hỏi, điểm, thời gian và tốc độ game.

Kiến trúc thật của repo:

- Frontend game: HTML/CSS/JavaScript thuần
 - Render 3D: `Three.js` + `WebGL`
 - Hero: model `glTF` có animation
 - Model phụ: `STL` cho cloud/ring, `Collada` cho background enemy
 - Backend: `Node.js` + `Express`
 - Database: `SQLite` qua `better-sqlite3`
 - Auth admin: `JWT` + `bcrypt` + `cookie` `HttpOnly`
 - PWA/offline: `manifest` + `service worker`
 - Test: `node:test` + `supertest`
-

3. Những điểm lệch giữa báo cáo và source hiện tại

Khi đối chiếu `BaoCao_SonicMathRunner.docx` với repo hiện tại, có vài khác biệt quan trọng:

Nội dung	Báo cáo Word	Source hiện tại
Số câu hỏi	13.800+	Seed thực tế là 1.200 câu: lớp 6 có 100, lớp 7 có 100, lớp 8 có 1.000
Cơ chế điều khiển	mô tả có nhảy, trái/phải	source hiện tại chỉ thấy đổi lane trái/phải, không có logic nhảy
AI	báo cáo xếp vào sản phẩm ứng dụng AI	repo hiện tại không có mô hình AI, API AI, ML pipeline, hay tích hợp LLM
Theo dõi tiến độ	mô tả như tính năng hệ thống	source hiện tại lưu lịch sử câu đã hiện/trả lời ở <code>localStorage</code> trên từng trình duyệt, không lưu server

Vì vậy, nếu dùng repo này làm chuẩn kỹ thuật, cần mô tả theo source hiện tại chứ không nên lặp lại nguyên văn báo cáo Word.

4. Kiến trúc tổng thể của hệ thống

4.1. Luồng người chơi

- Mở `index.html`
- `index.html` tải `EndlessRunner.htm`
- `EndlessRunner.htm` nạp:
 - `EndlessRunner.js` chứa `Three.js`, loaders, model, texture, audio dạng embed
 - `shared/questionModel.js`
 - `questionBank.js`
- Người chơi chọn lớp `lop6`, `lop7`, `lop8`
- Frontend gọi API `GET /api/levels/:level/question-bank`
- Game dựng scene 3D, bắt đầu chạy
- Khi va vào `ring`, game dừng tạm và mở quiz overlay
- Trả lời đúng thì cộng điểm, sai hoặc hết giờ thì mất tim
- Hết tim thì `game over`

4.2. Luồng quản trị

- Mở `admin.html`

2. Đăng nhập bằng mật khẩu admin
3. Backend kiểm tra mật khẩu bằng `bcrypt.compare`
4. Nếu đúng, server phát JWT và lưu vào cookie `HttpOnly`
5. Admin có thể:
 - xem question bank theo lớp
 - thêm/sửa/xóa câu hỏi
 - đổi điểm theo độ khó
 - đổi thời gian theo độ khó
 - đổi tốc độ game
 - reset lịch sử câu đã trả lời trên trình duyệt hiện tại

4.3. Phân ranh frontend/backend

- Frontend chịu trách nhiệm render, gameplay, UI quiz, hiệu ứng, âm thanh, điều khiển, local progress.
 - Backend chịu trách nhiệm xác thực admin, cấp question bank, ghi SQLite, cập nhật cấu hình toàn cục.
-

5. Cấu trúc source code

5.1. Frontend game

- `index.html` : trang vào, chỉ có loader và fetch `EndlessRunner.htm`
- `EndlessRunner.htm` : shell HTML chính của game, chứa UI, overlay, game loop script
- `EndlessRunner.js` : bundle asset và thư viện 3D/audio
- `worker.js` : service worker cache offline
- `EndlessRunner.json` : web app manifest

5.2. Frontend admin

- `admin.html` : toàn bộ giao diện quản trị bằng HTML/CSS/JS thuần
- `shared/questionModel.js` : model kiểm tra dữ liệu câu hỏi dùng chung cho client/server
- `questionBank.js` : client-side layer gọi API, cache dữ liệu, lưu progress local

5.3. Backend

- `server/index.js` : boot server
- `server/app.js` : tạo Express app, route API, static serving, error handling
- `server/db.js` : schema SQLite, seed dữ liệu, thao tác CRUD
- `server/auth.js` : JWT, cookie, bcrypt
- `server/config.js` : resolve config và validate `.env`
- `server/scripts/hash-password.js` : tạo hash mật khẩu admin

5.4. Dữ liệu

- `questions/lop6.json`
- `questions/lop7.json`
- `questions/lop8.json`
- `.runtime/game-sonic-running.sqlite`

5.5. Test

- `test/server.test.js`
- `test/config.test.js`

6. Công nghệ được chọn và lý do chọn

6.1. HTML/CSS/JavaScript thuần

Xác nhận từ source

Lý do hợp với dự án này:

- nhẹ, không cần build pipeline phức tạp
- dễ mở trực tiếp trong browser
- kiểm soát tốt performance cho game nhỏ
- admin page và game page có thể viết nhanh bằng DOM API

Nhược điểm:

- file `EndlessRunner.htm` rất dài
- khó bảo trì khi game lớn lên

- asset đang bị nhúng cứng vào bundle

6.2. Three.js + WebGL

Xác nhận từ source

Repo dùng `THREE.WebGLRenderer`, `GLTFLoader`, `STLLoader`, `ColladaLoader`, `AnimationMixer`.

Lý do chọn:

- render 3D trực tiếp trên web
- API dễ hơn viết WebGL thuần
- hỗ trợ nhiều định dạng model
- đủ mạnh cho game runner 3 lane, scene đơn giản, ít nhân vật

6.3. Node.js + Express

Xác nhận từ source

Lý do chọn:

- đồng nhất JavaScript ở cả frontend và backend
- API CRUD nhỏ, Express là đủ
- nhanh để làm trang admin và REST API

6.4. SQLite

Xác nhận từ source

Lý do chọn:

- không cần cài server DB riêng
- phù hợp ứng dụng nhỏ hoặc demo thi sáng tạo
- dễ seed từ JSON
- rất tiện khi admin chỉ thao tác câu hỏi, điểm, thời gian, tốc độ

6.5. JWT + bcrypt

Xác nhận từ source

Lý do chọn:

- `bcrypt` để hash mật khẩu admin
- `JWT` để giữ phiên đăng nhập admin
- cookie `HttpOnly` giảm rủi ro lộ token qua JavaScript client

6.6. PWA

Xác nhận từ source

Repo có:

- `EndlessRunner.json`
- `worker.js`
- cache asset tĩnh và API question bank

Lý do chọn:

- tăng cảm giác "app-like"
- chơi được tốt hơn trong điều kiện mạng yếu
- hỗ trợ cài lên điện thoại/máy tính bảng

7. Cách game được triển khai ở mức kỹ thuật

7.1. Scene 3D

Game dựng các thành phần chính:

- road: `BoxGeometry` + texture stone lặp
- grass hai bên: `BoxGeometry` + texture grass lặp
- ocean nền: `BoxGeometry` + texture nước
- sky: gradient vẽ bằng `canvas`
- cloud: model `STL`
- background enemy: model `Collada`
- hero: model `glTF` có animation
- obstacle:
 - `ring` là model `STL`
 - `enemy` là hình học dựng bằng code từ sphere + cone

7.2. Hero

Xác nhận từ source

Hero được load bằng:

- `new THREE.GLTFLoader().parse(sonicModel, ...)`
- scale lớn lên `11x`
- dùng `AnimationMixer`
- phát animation đầu tiên trong file glTF

Điều này cho thấy asset nhân vật chính là model 3D có rig/animation sẵn.

7.3. Background enemy

Xác nhận từ source

Background enemy được load bằng:

- `new THREE.ColladaLoader().parse(robotnikModel)`

Trong dữ liệu có các tên material như:

- `edge_color...`
- `Translucent_Glass_Blue`
- `skp_material`

Suy luận hợp lý: model này có khả năng từng được export từ `SketchUp` hoặc một pipeline tương thích Collada có dấu vết `skp_material`.

7.4. Ring và cloud

Xác nhận từ source

Ring và cloud được load bằng `STLLoader`.

Điều này cho thấy 2 asset này là mesh tĩnh, không cần rig.

7.5. Obstacle enemy dựng bằng code

Xác nhận từ source

Obstacle kẻ địch không dùng file model riêng, mà được tạo trực tiếp bằng:

- 1 sphere làm thân
- 4 cone làm gai
- merge geometry

Đây là một mẹo rất thực dụng:

- giảm công làm asset
- giảm số file ngoài
- phù hợp game runner đơn giản

7.6. Điều khiển

Xác nhận từ source

Game hiện tại hỗ trợ:

- bàn phím: `A/D` hoặc mũi tên trái/phải
- mobile: swipe trái/phải
- quiz: bấm `A/B/C/D` hoặc click/touch

Không thấy logic nhảy trong source hiện tại.

7.7. Va chạm

Xác nhận từ source

Va chạm dùng:

- `THREE.Box3().setFromObject(hero)`
- `THREE.Box3().setFromObject(obstacle)`
- `intersectsBox(...)`

Nếu chạm `ring` thì mở quiz. Nếu chạm obstacle thưởng thì trừ tim.

7.8. Hệ tim, điểm, game over

Xác nhận từ source

- Điểm tăng theo `question.point`
- Người chơi có `3` tim
- Sai/hết giờ hoặc bỏ lỡ ring đều có thể mất tim

- Hết tim thì `gameOver()`
- High score lưu ở cookie `highscoresonic`

7.9. Quiz integration

Xác nhận từ source

Khi chạm ring:

1. Lấy câu hỏi kế tiếp từ hàng đợi
2. Dừng gameplay bằng `pauseDueToQuiz`
3. Hiện overlay
4. Bắt đầu countdown
5. Đúng thì cộng điểm
6. Sai/hết giờ thì trừ tim
7. Ẩn overlay và game chạy tiếp

7.10. Tiến độ câu hỏi

Xác nhận từ source

`questionBank.js` lưu tiến độ ở `localStorage` :

- câu đã hiện
- số lần đã hiện
- trạng thái `shown/correct/wrong/timeout`
- thời điểm trả lời cuối

Điểm cần lưu ý:

- đây là lưu cục bộ theo từng trình duyệt
- không phải progress đồng bộ server

8. Thiết kế dữ liệu câu hỏi

Mỗi câu hỏi có cấu trúc:

```

{
  "id": "6q001",
  "difficulty": "easy",
  "question": "9 × 7 = ?",
  "answers": {
    "A": "63",
    "B": "56"
  },
  "correctAnswer": "A",
  "point": 10,
  "time": 12
}

```

Rule validate từ `shared/questionModel.js`:

- `id` bắt buộc duy nhất
- `question` không rỗng
- phải có ít nhất 2 đáp án
- đáp án phải khai báo theo thứ tự `A -> B -> C -> D`, không được thủng giữa chừng
- `correctAnswer` phải nằm trong các đáp án có thật
- `point` ≥ 0
- `time` ≥ 1 và phải là số nguyên

Số lượng seed hiện tại:

- `lop6`: 100
- `lop7`: 100
- `lop8`: 1.000
- tổng: 1.200

Phân bố độ khó:

- lớp 6: 40 easy, 30 medium, 20 hard, 10 expert
- lớp 7: 40 easy, 30 medium, 20 hard, 10 expert
- lớp 8: 400 easy, 300 medium, 200 hard, 100 expert

9. Cấu trúc database SQLite

Xác nhận từ source

Repo tạo 3 bảng chính:

9.1. `questions`

Lưu:

- level
- id
- sort_order
- difficulty
- question
- answer_a, answer_b, answer_c, answer_d
- correct_answer
- point
- time
- created_at
- updated_at

Khóa chính:

- `(level, id)`

9.2. `difficulty_settings`

Lưu cấu hình mặc định theo độ khó:

- `default_point`
- `default_time`

9.3. `level_settings`

Lưu:

- `game_speed`

9.4. Cách seed dữ liệu

Lần khởi động đầu tiên:

- nếu bảng `questions` rỗng
- server đọc `questions/lop6.json`, `lop7.json`, `lop8.json`
- validate
- insert vào SQLite

Nếu DB đã có dữ liệu thì JSON không tự overwrite.

10. API backend

Xác nhận từ source

Public API

- `GET /api/health`
- `GET /api/levels/:level/question-bank`

Admin auth API

- `POST /api/admin/login`
- `POST /api/admin/logout`
- `GET /api/admin/session`

Admin data API

- `PUT /api/levels/:level/questions`
- `PUT /api/levels/:level/settings/point`
- `PUT /api/levels/:level/settings/time`
- `PUT /api/levels/:level/settings/speed`

Điểm đáng chú ý:

- thay đổi point/time/speed đang áp dụng cho toàn bộ 3 lớp, không chỉ lớp đang mở
- API được bảo vệ bởi middleware `requireAdminAuth`

11. Công cụ nên dùng để lập trình dự án này

Phần này gồm 2 lớp: công cụ chắc chắn đang dùng trong repo, và công cụ nên dùng nếu tái tạo dự án.

11.1. Công cụ chắc chắn có trong repo

- `Node.js`
- `npm`
- `Express`
- `SQLite`
- `better-sqlite3`
- `bcrypt`
- `jsonwebtoken`
- `supertest`
- browser có hỗ trợ `WebGL`, `service worker`, `AudioContext`

11.2. Công cụ lập trình nên dùng khi tái tạo

Suy luận hợp lý

- `VS Code` hoặc editor tương đương để code JavaScript/HTML/CSS
- `Chrome DevTools` để debug render, network, localStorage, service worker
- `DB Browser for SQLite` để xem và kiểm tra database
- `Git` để quản lý version
- `Postman` hoặc `Insomnia` để test API admin
- `npm` scripts để chạy local

11.3. Công cụ làm UI/asset 2D

Suy luận hợp lý

- `Figma` cho wireframe và flow giao diện
- `Photoshop`, `GIMP` hoặc `Krita` để chỉnh icon, ảnh share, favicon, texture
- `Audacity` để cắt/chỉnh âm thanh nếu cần

12. Dùng công cụ gì để làm nhân vật và asset 3D

Đây là phần quan trọng nhất đối với câu hỏi “dùng gì để làm nhân vật”.

12.1. Điều xác nhận được từ source

- Sonic được nạp dưới dạng `glTF`, có animation
- Ring được nạp bằng `STL`
- Cloud được nạp bằng `STL`
- Background Robotnik được nạp bằng `Collada`
- Obstacle enemy nhỏ được dựng bằng code, không cần phần mềm 3D ngoài

12.2. Điều không xác nhận được hoàn toàn

Repo không chứa:

- file `.blend`
- file `.glb/.gltf` gốc tách riêng
- file `.dae` gốc tách riêng
- file nguồn texture rời

Tất cả đang bị nhúng thẳng vào `EndlessRunner.js`.

Vì vậy:

- không thể khẳng định chính xác phần mềm dựng Sonic ban đầu là gì
- không thể khẳng định 100% ring/cloud được dựng bằng Blender hay phần mềm CAD khác
- nhưng có thể kết luận chắc rằng pipeline đầu ra dùng `glTF`, `STL`, `Collada`

12.3. Pipeline hợp lý để làm lại nhân vật

Suy luận hợp lý

Nhân vật chính Sonic

Nên dùng `Blender` vì:

- tạo model low-poly dễ

- rig và animation tốt
- export `glTF 2.0` trực tiếp rất phù hợp cho Three.js

Quy trình:

1. dựng mesh low-poly
2. UV unwrap
3. tô texture hoặc vật liệu đơn giản
4. rig xương
5. tạo animation chạy
6. export `glTF/GLB`
7. load bằng `THREE.GLTFLoader`

Ring và cloud

Có thể dùng `Blender` để dựng mesh tĩnh rồi export `STL`.

Nếu làm lại hôm nay, thực tế hơn là export `glTF` hoặc giữ dạng geometry code, vì `STL` không mang vật liệu/animation tốt bằng `glTF`.

Robotnik/background object

Vì source có dấu vết `skp_material`, `edge_color`, nên nhiều khả năng asset này từng đi qua `SketchUp` hoặc workflow tương tự rồi export `Collada (.dae)`.

Nếu tái tạo:

- có thể tiếp tục dùng `SketchUp -> Collada`
- hoặc tốt hơn: gom về `Blender -> glTF` để thống nhất pipeline

Obstacle enemy nhỏ

Không nhất thiết dùng Blender. Repo hiện tại dựng hoàn toàn bằng code:

- thân cầu
- 4 gai hình nón
- merge geometry

Đây là giải pháp rất phù hợp cho prototype hoặc game giáo dục nhỏ.

13. Cách tạo lại trò chơi này từ đầu

Bước 1. Chọn phạm vi và gameplay

Chốt trước:

- game runner 3 lane
- người chơi né obstacle
- chạm ring thì bật quiz
- đúng cộng điểm, sai trừ tim
- chọn level theo lớp học

Bước 2. Dựng skeleton project

Tối thiểu nên có:

```
/
  index.html
  EndlessRunner.htm
  EndlessRunner.js
  admin.html
  questionBank.js
  shared/questionModel.js
  server/
  questions/
  worker.js
  EndlessRunner.json
```

Bước 3. Làm backend trước

Vì quiz phụ thuộc dữ liệu, nên nên làm backend sớm:

1. tạo Express app
2. tạo SQLite schema
3. viết validate question model
4. viết API lấy question bank

- viết API admin login
- viết API cập nhật question/settings

Bước 4. Làm định dạng dữ liệu câu hỏi

Chuẩn hóa:

- id
- difficulty
- question
- answers
- correctAnswer
- point
- time

Đây là phần cực quan trọng vì quiz chỉ ổn khi dữ liệu được validate chặt.

Bước 5. Làm game shell 3D

- khởi tạo `THREE.WebGLRenderer`
- tạo camera
- dựng road, grass, ocean
- thêm light
- thêm sky gradient
- thêm background props

Bước 6. Tích hợp hero

- dựng model trong Blender
- rig và animate
- export `glTF`
- load bằng `GLTFLoader`
- dùng `AnimationMixer` để phát animation chạy

Bước 7. Tạo obstacle system

- định nghĩa 3 vị trí lane
- spawn obstacle ở xa

3. cho obstacle chạy về phía camera
4. recycle obstacle khi vượt màn hình
5. random lane nhưng tránh lặp lane liên tục

Bước 8. Tạo quiz system

1. lấy question bank theo level
2. shuffle queue
3. khi ring bị chạm thì bật overlay
4. countdown theo `question.time`
5. xử lý đúng/sai/timeout
6. cập nhật score và lives
7. ghi progress local

Bước 9. Tạo admin page

Admin page nên có:

- login
- level switcher
- form tạo/sửa câu hỏi
- bảng danh sách câu hỏi
- chỉnh point theo difficulty
- chỉnh time theo difficulty
- chỉnh game speed
- reset progress local

Bước 10. Làm âm thanh

Repo hiện tại dùng `Web Audio API` :

- nhạc nền
- nhạc game over
- ring sound
- hurt sound
- correct sound

Nếu làm lại:

- có thể giữ Web Audio API
- hoặc dùng thư viện âm thanh nhẹ nếu muốn code dễ hơn

Bước 11. Làm PWA

1. tạo `manifest`
2. viết `service worker`
3. cache static assets
4. cache question bank GET API
5. fallback offline

Bước 12. Viết test

Ít nhất nên test:

- config secret
- seed database
- health endpoint
- login/logout
- update question bank
- update point/time/speed
- persistence sau restart

Bước 13. Triển khai

Repo hiện có `start.sh` và `restart.sh` cho hướng triển khai Debian/Ubuntu với:

- Node.js
- PM2
- Nginx
- SSL

Nếu triển khai lại, đây là hướng practical cho demo hoặc sản phẩm nhỏ.

14. Lệnh vận hành hiện tại

Cài dependency

```
npm install
```

Tạo hash mật khẩu admin

```
npm run hash-password -- your-admin-password
```

Tạo `.env`

```
PORT=3000  
JWT_SECRET=replace-with-a-long-random-secret  
ADMIN_PASSWORD_HASH=paste-generated-hash-here
```

Chạy app

```
npm start
```

Chạy test

```
npm test
```

15. Khuyến nghị nếu muốn nâng cấp kiến trúc

Nếu tiếp tục phát triển dự án này, nên cân nhắc:

1. Tách asset ra khỏi `EndlessRunner.js`, không nhúng toàn bộ base64 vào một file khổng lồ.
2. Chuẩn hóa toàn bộ model về `glTF` thay vì trộn `glTF + STL + Collada`.
3. Tách game logic thành nhiều module thay vì dồn trong `EndlessRunner.htm`.
4. Chuyển progress câu hỏi từ `localStorage` lên server nếu muốn theo dõi người dùng thật.
5. Thêm cơ chế analytics hoặc leaderboard thật thay vì chỉ cookie local.
6. Nếu báo cáo muốn nhấn mạnh AI, cần bổ sung tính năng AI thật, ví dụ sinh câu hỏi, chấm thích nghi độ khó, hoặc trợ lý học tập.

16. Kết luận

Nếu nhìn đúng theo source hiện tại, `Sonic Math Runner` là một sản phẩm web game giáo dục 3D được xây bằng stack rất thực dụng:

- `Three.js` để làm game 3D
- `HTML/CSS/JS thuần` để giữ hệ thống nhẹ
- `Node.js + Express + SQLite` để quản lý dữ liệu và admin
- `JWT + bcrypt` để bảo vệ admin
- `PWA` để chạy tốt trên thiết bị di động

Phần "làm nhân vật bằng gì" không thể xác nhận 100% từ repo vì file nguồn 3D gốc không còn được lưu riêng, nhưng pipeline đầu ra là rất rõ:

- nhân vật chính: `glTF` có animation
- asset tĩnh: `STL`
- background object: `Collada`

Nếu phải tái tạo dự án này một cách bài bản hôm nay, lựa chọn thực tế nhất là:

- `Blender` để dựng/rig/export nhân vật và mesh 3D
- `Figma` hoặc công cụ tương đương để thiết kế UI
- `Photoshop/GIMP/Krita` để làm texture/icon
- `Audacity` để xử lý âm thanh
- `VS Code + Chrome DevTools + SQLite Browser` để phát triển và debug